Diseño y desarrollo de servicios web - caso.

Evidencia:

GA7-220501096-AA5-EV01

Presentado por:

Ronald Fabian Mendoza Rodríguez

Juan Sebastián Ávila Patiño

Servicio Nacional de Aprendizaje SENA

Tecnólogo en Análisis y Desarrollo de Software

Ficha:

3235898

2026

INTRODUCCIÓN

Los servicios web REST permiten que diferentes aplicaciones intercambien información mediante solicitudes HTTP y respuestas estructuradas. Esta evidencia presenta el desarrollo de un servicio de registro y autenticación de usuarios con Java y Spring Boot. La aplicación expone endpoints para registrar usuarios, validar credenciales y comunicar los resultados mediante códigos de estado HTTP y mensajes de respuesta.

El documento describe la estructura técnica reportada del proyecto, las rutas implementadas, las validaciones, las medidas de protección de contraseñas, las pruebas automatizadas y el repositorio de control de versiones. Las capturas de pantalla deben incorporarse en los espacios señalados una vez tomadas directamente del proyecto.

Objetivos

- Diseñar e implementar un servicio web REST para el registro y la autenticación de usuarios.
- Validar los datos recibidos y responder de forma coherente ante solicitudes correctas e incorrectas.
- Proteger las contraseñas mediante el uso de BCrypt, evitando almacenarlas en texto plano.
- Organizar el código en capas y componentes de Spring Boot para facilitar su mantenimiento.
- Gestionar el código fuente con Git y publicarlo en un repositorio de GitHub.

TECNOLOGÍAS Y HERRAMIENTAS

Tecnología / herramienta	Uso en el proyecto
Java 21	Lenguaje y entorno de ejecución reportado.
Spring Boot 2.7.18	Configuración y ejecución de la aplicación web.
Spring Web / REST	Exposición de endpoints HTTP.
Spring Data JPA	Acceso a los datos mediante repositorio.
H2	Base de datos configurada para el proyecto y pruebas.
Spring Security Crypto / BCrypt	Codificación segura de contraseñas.
Maven Wrapper	Ejecución de tareas Maven sin requerir Maven instalado globalmente.
JUnit / Spring Test / MockMvc	Pruebas automatizadas de los endpoints.
Git y GitHub	Control de versiones y publicación del código.

ARQUITECTURA Y COMPONENTES PRINCIPALES

Según la versión actual reportada del proyecto, la aplicación está organizada en los siguientes componentes:

- ServicioAutenticacionApplication.java: punto de entrada de Spring Boot.
- AuthController.java: recibe las solicitudes REST de registro y autenticación.
- AuthService.java: concentra la lógica de negocio para registrar usuarios y validar credenciales.
- Usuario.java: entidad que representa al usuario persistido.
- UsuarioRepository.java: operaciones de persistencia mediante Spring Data JPA.
- AuthRequest.java: estructura de los datos recibidos en las solicitudes.
- ApiResponse.java: estructura común para los mensajes de respuesta.
- GlobalExceptionHandler.java: manejo centralizado de errores de validación, JSON y persistencia.
- SecurityConfig.java: configuración relacionada con seguridad y codificación de contraseñas.

La configuración de pruebas utiliza una base H2 en memoria independiente para evitar que las pruebas automatizadas modifiquen los datos de desarrollo.

ENDPOINTS DEL SERVICIO

Registro de usuario

Método y ruta: POST /api/auth/registro

Función: recibe los datos de registro, valida la solicitud, verifica si el usuario ya existe y guarda la contraseña codificada con BCrypt.

Respuestas reportadas:

- 201 Created: {"mensaje":"Registro satisfactorio."}
- 400 Bad Request: datos inválidos, campos requeridos ausentes o cuerpo JSON mal formado.
- 409 Conflict: {"mensaje":"El usuario ya está registrado."}

Inicio de sesión

Método y ruta: POST /api/auth/login

Función: verifica que el usuario exista y compara la contraseña recibida con el hash almacenado.

Respuestas reportadas:

- ✓ 200 OK: {"mensaje":"Autenticación satisfactoria."}
- ✓ 401 Unauthorized: {"mensaje":"Error en la autenticación."} cuando las credenciales no son válidas o el usuario no existe.

Nota: los nombres exactos de los campos del JSON deben confirmarse con AuthRequest.java antes de pegar ejemplos de solicitudes. No se deben incluir contraseñas reales ni datos personales en las capturas.

Validaciones y seguridad

- ➢ Validación de los datos de entrada y manejo de solicitudes incompletas o mal formadas.
- ➢ Control de usuarios duplicados durante el registro.
- ➢ Codificación de contraseñas con BCrypt antes de persistirlas.
- ➢ Respuesta de autenticación no satisfactoria cuando las credenciales no coinciden.
- ➢ Manejo centralizado de excepciones para mantener respuestas HTTP consistentes.
- ➢ Separación de la base de datos de pruebas respecto de la configuración de desarrollo.

Pruebas automatizadas

De acuerdo con el resultado de ejecución informado durante el desarrollo, el comando

```
.\mvnw.cmd clean test
```

finalizó con BUILD SUCCESS y 9 pruebas ejecutadas, sin fallos, errores ni pruebas omitidas. Los escenarios reportados cubren carga del contexto, registro correcto, registro duplicado, entrada inválida, inicio de sesión correcto, contraseña incorrecta, usuario inexistente, almacenamiento de contraseña con BCrypt y cuerpo JSON mal formado o vacío.

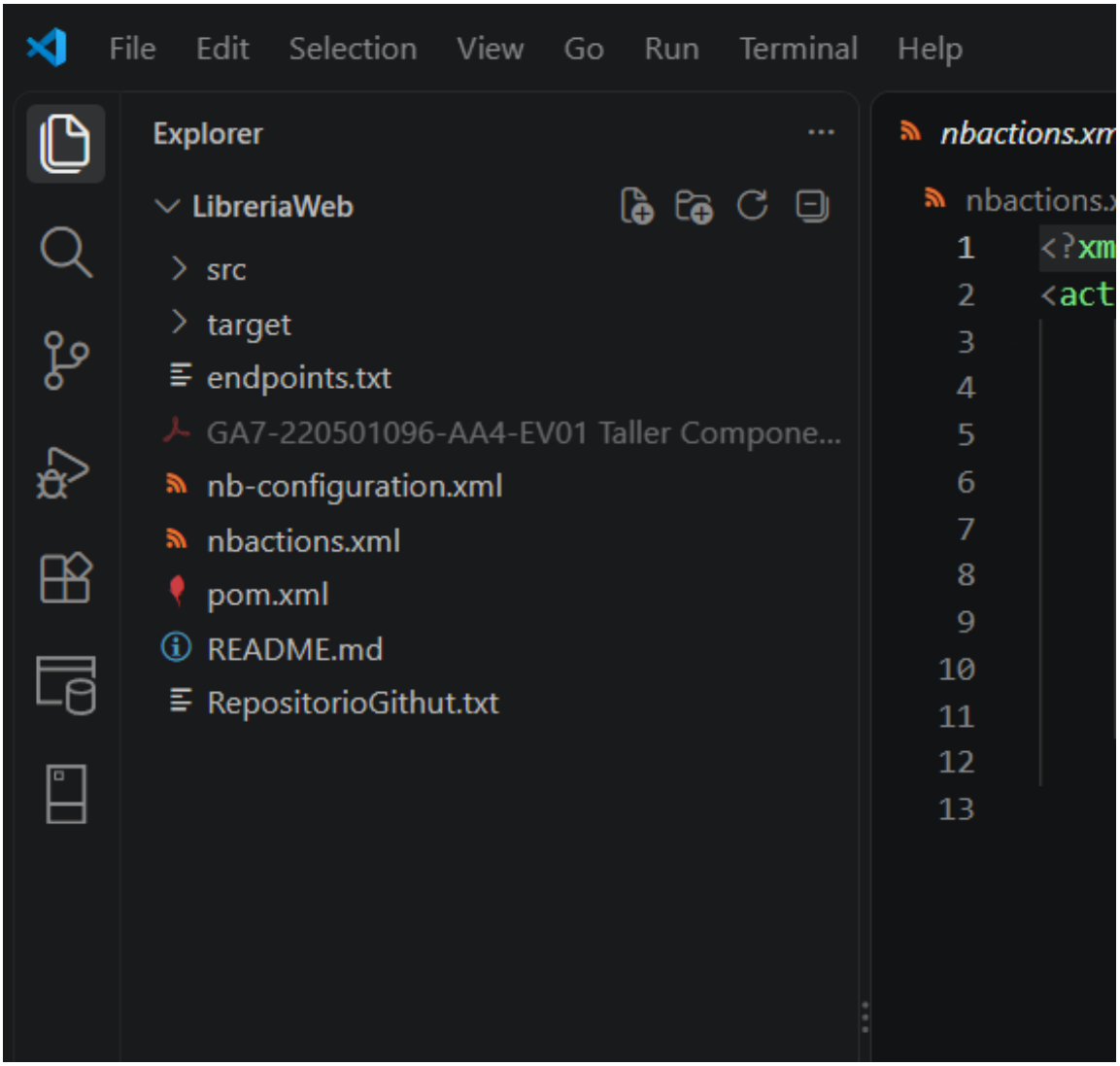
Antes de entregar, se recomienda volver a ejecutar el comando en el equipo del grupo y reemplazar o complementar esta descripción con una captura real de la consola de Maven que muestre el resultado final.

Control de versiones y repositorio

Repositorio GitHub del proyecto:

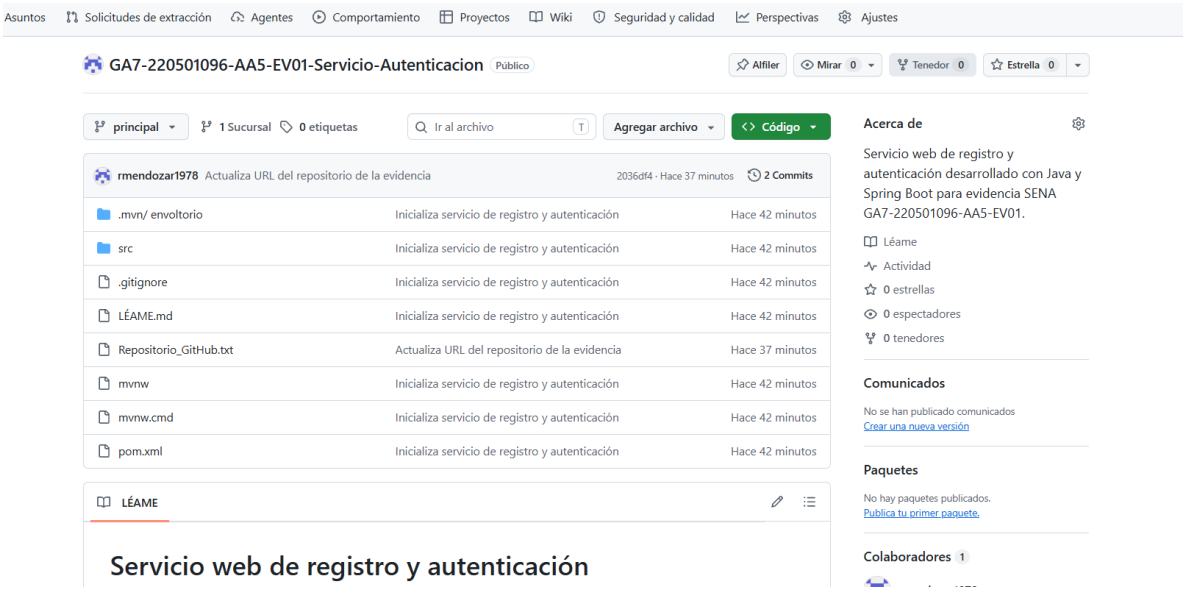
https://github.com/rmendozar1978/GA7-220501096-AA5-EV01-Servicio-Autenticacion

El proyecto cuenta con historial de Git y una rama principal publicada en GitHub, según el estado reportado. Estructura del proyecto



Repositorio GitHub

Página del repositorio con el nombre y la lista de archivos; que se vea la URL en el navegador.



Conexión y Repositorios (Backend con Spring Boot / Java y JPA)

El sistema emplea Spring Data JPA y repositorios administrados para interactuar de forma automatizada con la base de datos MySQL, asegurando una capa de persistencia limpia:

```
package com.librerialuz.repository;

import com.librerialuz.repository;
import com.librerialuz.jpan .UsuarioRepository;


/**
 * @author Juan Sebastian Avila Patiño
 */
@Repository
public interface UsuarioRepository extends
     Long > JpaRepository<Usuario,
    Usuario findByCorreo(String "Correo");



    Usuario.RepositoryfindByCorreo(String "Correo");
}
```

```
package com.librerialuz.controller;

import com.librerialuz.Usuario;
import com.librerialuz.RegstrarUsuarior;
import com.librerialuz.AuthContoller;
import com.librerialuz.AuthContoion(suistasa Rodríguez;

/**
 * @author Ronald Fabian Mendoza Rodríguez
 */
@RestController
@RequestMapping("Auth//bin")
@CrossOrigin(Usuario)
public class AuthController {

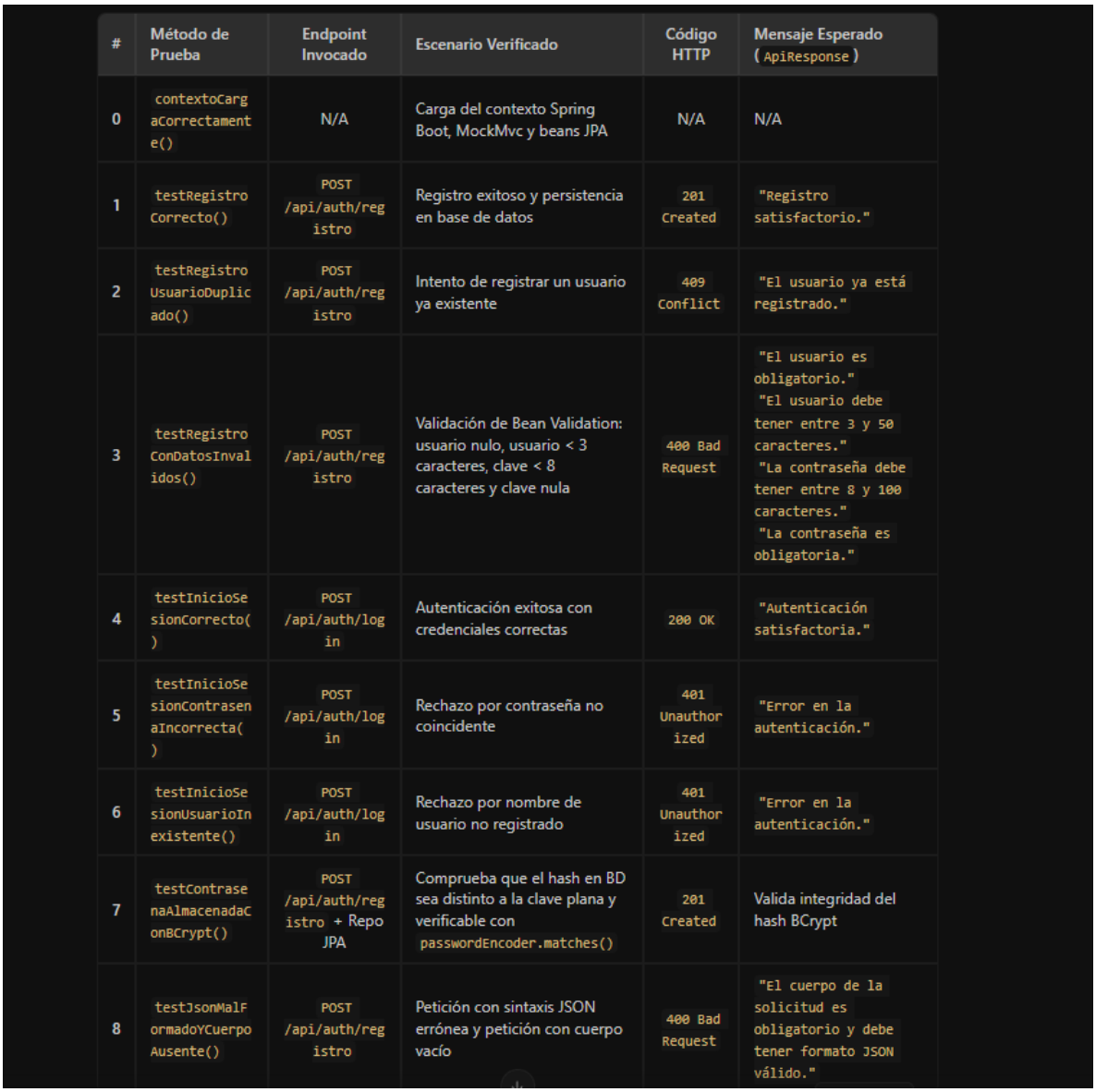
    @Autowired
    Usuario Repository= UsuarioRepository;

    /**
     * @author Usuario coms to registrarUsuario
     */
    public void registrarUsuario(Usuario StringGomer, String) {
        UsuarioReposicrycoreo.registrarUsuario(findByCorreo));
    }

    /**
     * @author InciarSesion comen iniciarSesion or
     * @crossOrigin
     */
    public void iniciarSesion(Usuario StringObject Sesion) {
        return publicRestion(iniciarSesion);
    }
}
```

Manejo de errores

GlobalExceptionHandler.java mostrando el tratamiento de validaciones y excepciones.



#	Método de Prueba	Endpoint Invocado	Escenario Verificado	Código HTTP	Mensaje Esperado (`ApiResponse`)
0	`contextoCargaCorrectamente()`	N/A	Carga del contexto Spring Boot, MockMvc y beans JPA	N/A	N/A
1	`testRegistroCorrecto()`	`POST /api/auth/registro`	Registro exitoso y persistencia en base de datos	`201 Created`	`"Registro satisfactorio."`
2	`testRegistroUsuarioDuplicado()`	`POST /api/auth/registro`	Intento de registrar un usuario ya existente	`409 Conflict`	`"El usuario ya está registrado."`
3	`testRegistroConDatosInvalidos()`	`POST /api/auth/registro`	Validación de Bean Validation: usuario nulo, usuario < 3 caracteres, clave < 8 caracteres y clave nula	`400 Bad Request`	`"El usuario es obligatorio."` `"El usuario debe tener entre 3 y 50 caracteres."` `"La contraseña debe tener entre 8 y 100 caracteres."` `"La contraseña es obligatoria."`
4	`testInicioSesionCorrecto()`	`POST /api/auth/login`	Autenticación exitosa con credenciales correctas	`200 OK`	`"Autenticación satisfactoria."`
5	`testInicioSesionContrasenaIncorrecta()`	`POST /api/auth/login`	Rechazo por contraseña no coincidente	`401 Unauthorized`	`"Error en la autenticación."`
6	`testInicioSesionUsuarioInexistente()`	`POST /api/auth/login`	Rechazo por nombre de usuario no registrado	`401 Unauthorized`	`"Error en la autenticación."`
7	`testContrasenaAlmacenadaConBCrypt()`	`POST /api/auth/registro` + Repo JPA	Comprueba que el hash en BD sea distinto a la clave plana y verificable con `passwordEncoder.matches()`	`201 Created`	Valida integridad del hash BCrypt
8	`testJsonMalFormadoYCuerpoAusente()`	`POST /api/auth/registro`	Petición con sintaxis JSON errónea y petición con cuerpo vacío	`400 Bad Request`	`"El cuerpo de la solicitud es obligatorio y debe tener formato JSON válido."`

Conclusiones

- El proyecto implementa una API REST orientada al registro y autenticación de usuarios mediante Spring Boot. La separación entre controlador, servicio, entidad y repositorio facilita la organización del código, mientras que las respuestas HTTP permiten comunicar resultados de operación y errores de forma comprensible.
- El uso de BCrypt mejora la protección de las contraseñas almacenadas. Las pruebas automatizadas reportadas cubren escenarios positivos y negativos de registro y autenticación. El repositorio GitHub aporta trazabilidad y respaldo del desarrollo. Para fortalecer la evidencia, deben incluirse las capturas reales indicadas y verificarse los resultados finales antes de entregar.